

Stack Tecnológico Recomendado

Para um **POC web** de sistema de avaliação imobiliária com IA em jul/2025, recomendamos um *stack* aberto e moderno que equilibre adoção de mercado e agilidade de desenvolvimento. Em geral, usa-se **JavaScript/Python** como base: React (ou Vue) no front-end e Node.js ou Python (Django/FastAPI) no back-end ¹ ². Por exemplo, PixelBrainy destaca que “as linguagens dominantes no *stack* web/IA são Python e JavaScript (React/Node)” ¹. Uma arquitetura MERN (MongoDB, Express, React, Node.js) é amplamente adotada e escalável ³. Para prototipagem rápida, frameworks Python como **Django** ou **FastAPI** são excelentes – Python é de fácil leitura e conta com inúmeras bibliotecas (incluindo ML) ². O *stack* escolhido deve incluir também:

- **Banco de dados:** PostgreSQL com extensão PostGIS para consultas geoespaciais (zonas, vizinhança, proximidade) – como demonstrado, PostGIS reduz complexidade de infraestrutura e permite consultas espaciais em milissegundos ⁴ ⁵. Em complemento, um NoSQL (como MongoDB) pode agilizar protótipo.

- **Mapas e geocodificação:** Leaflet.js + OpenStreetMap (BSD, zero dependências) para mapas interativos – é a biblioteca de mapeamento mais usada em 2025. Usar **Nominatim** (OSM) ou Google Maps API (com custos) para geocodificação de endereços.

- **Visão Computacional:** OpenCV e modelos YOLO (doUltralytics) ou Detectron para análise de fotos – YOLO é rápido e “one-stage” (uma única passada) para detecção de objetos ⁶. Ele identifica características (fendas, acabamentos, móveis, vagas, etc.) a partir das fotos, classificando padrão e estado do imóvel.

- **IA e LLM:** Empregar Python com bibliotecas de aprendizado (TensorFlow, PyTorch, scikit-learn) para regressão estatística, e **LangChain** ou bibliotecas similares para orquestração de LLMs (GPT-4, Claude, etc.) ⁷ ⁸. LangChain facilita chamar a API de modelos de linguagem, mantendo histórico e estado. Para geração dinâmica de perguntas, LLMs (por exemplo GPT-4 da OpenAI ou Google Gemini) são usados via API. Segundo ImaginaryCloud, “AI-First stacks usando Python, LangChain e APIs OpenAI são ideais para aplicações inteligentes” ⁸.

- **Ferramentas de Desenvolvimento:** Usar Git/GitHub, Docker (containerização) e CI/CD (GitHub Actions/Jenkins). Para acelerar codificação, aproveitar **Gemini CLI** (Google) e **Claude Code** (Anthropic) – são IA de terminal que entendem o projeto e escrevem/ajustam código. Gemini CLI é *open-source* e grátis (basta conta Google) ⁹, enquanto Claude Code (pago) integra o modelo Claude a editores e CLI, automatizando mudanças em vários arquivos ¹⁰.

Figura: Exemplo de arquitetura multilayer com integração de IA generativa e dados imobiliários (camadas de diálogo, intent recognition, bancos de dados) ¹¹. A arquitetura geral do sistema deve ser em camadas: interface web (SPA React), API de back-end, módulos de IA/ML e banco de dados. Na camada de apresentação, React (ou Vue) gerencia formulários dinâmicos, mostra mapas (Leaflet) e carrega fotos. A API (Node/Express ou FastAPI) expõe endpoints REST para CRUD de dados e chamadas de IA. A seguir, detalhamos cada parte.

1. Front-end (Interface Web)

- **Framework:** React.js (preferencialmente com Next.js ou Vite) para criar SPA rápida e responsiva ³ ². Alternativas como Vue ou Angular são viáveis, mas React tem maior comunidade e ferramentas (Next.js, React Router, Redux/MobX). Para CSS, utilizar Tailwind ou Material UI para produtividade.

- **Navegação/Estado:** Next.js (SSR/SSR) ou React Router para rotas multi-etapa (wizard). Gerenciamento de estado via Redux/Context (respostas do usuário, lista de comparáveis, imagens).
- **Formulários Dinâmicos:** estruturar fluxo em etapas (endereço → fotos → questionário → resultado). Por exemplo, usar bibliotecas de formulários (Formik, React Hook Form) para lidar com validações. Conforme respostas do usuário, o front-end faz *fetch* dinâmico de novas perguntas via API.
- **Mapas:** integrar Leaflet.js para mostrar localização geográfica e vizinhança. Exibir mapa centrado no imóvel, marcadores em pontos de interesse (escolas, transporte, etc.), usando camadas de OSM ou Mapbox. Segundo dados de uso, Leaflet lidera com ~1.4M downloads/mês em 2025, sendo leve e extensível.
- **Upload de Fotos:** componente para múltiplos arquivos (mínimo ~20 fotos). Uso de previews (miniaturas). Ao subir, frontend envia imagens para a API (multipart/form-data), guardando-as temporariamente.

Figura: Arquitetura típica de um app React: componentes raiz comunicando-se com APIs e biblioteca de roteamento ³. A imagem ilustra o padrão MERN, onde uma SPA React consome APIs Express/Node. A alta adoção do MERN explica por que “MERN é considerado um dos melhores *full-stack* graças ao JavaScript uniforme e forte comunidade” ³. Na prática, nossa UI deve ser modular: criar componentes para cada etapa (endereço, fotos, perguntas). Utilizar validação instantânea nos campos e feedback visual. Garantir responsividade (Mobile-first) com CSS flexbox/grid e testes manuais nos principais navegadores.

2. Back-end (API e Processamento)

- **Linguagem/Framework:** Python (FastAPI) ou Node.js (Express/NestJS). Se o time está confortável com Python e foco em IA, FastAPI é excelente (alta performance, tipagem, fácil integração ML). Caso prefira JS, Express ou NestJS (TypeScript) são maduros. Ambos suportam exportação de rotas e middlewares (validação, autenticação). A decisão pode levar em conta bibliotecas necessárias: por exemplo, scikit-learn/pandas rodarão nativamente em Python.
- **Autenticação/Segurança:** JWT ou OAuth2 para API (login via certificado digital ICP-Brasil se desejado). Middleware para verificar permissões (apenas engenheiro agrimensor cadastrado pode assinar laudo).
- **Serviços de IA:** criar endpoints dedicados:
 - `/analyze/address`: recebe endereço → chama geocoding (Nominatim/Google) → retorna coordenadas, zona, etc.
 - `/fetch/comparables`: obtém comparáveis via scraping/API (ver abaixo) e retorna dados (próximos imóveis, preços).
 - `/analyze/images`: recebe fotos → executa visão computacional (detecção YOLO) → extrai atributos preliminares (número de cômodos, acabamentos, patologias).
 - `/generate/questions`: toma dados iniciais (endereço, tags de imagens) → invoca LLM via LangChain para criar perguntas personalizadas segundo NBR14653.
 - `/evaluate/insights`: após coleta de respostas, realiza cálculo de valor e fundamentação.
 - `/generate/report`: monta PDF final.
- **Raspagem de dados (Scraping):** para buscar comparáveis (ex.: sites como ZAP, OLX, VivaReal, registros públicos), usar Python (Requests + BeautifulSoup) ou Node (axios + Cheerio). Considerar APIs públicas (Ex.: **CPCON**, preçários, base do cartório, IBGE, cadastro imobiliário municipal) para zona de valor homogênea e zoneamento. Armazenar resultados em banco (caso similar). Por exemplo, script Python rastreando anúncios nos últimos 12 meses em raio 1km do imóvel.

- **Banco de Dados:** PostgreSQL com PostGIS para imóveis e transações históricas. Cada imóvel tem geometria (latitude/longitude) para consultas espaciais (p.ex. “encontre imóveis vendidos a <1km”). No PostGIS, usar funções como `ST_DWithin` e `ST_Distance` facilita buscar vizinhança ou aplicação de ZVH ⁴ ⁵. Aiven relata que PostGIS “resolveu elegantemente” desafios de busca espacial sem sistemas separados ⁴. MongoDB pode guardar documentos (ex.: respostas do questionário JSON).
- **Armazenamento de Arquivos:** fotos e PDFs podem ficar em armazenamento de objetos (AWS S3, MinIO) ou filesystem com disco local. Referenciar URL no banco.

3. Processamento de IA/ML

- **Visão Computacional (Fotos):** implementar pipeline de análise de imagens com YOLOv8 (Ultralytics) ou um modelo similar, possivelmente via Hugging Face. O modelo deve identificar padrões de acabamento, elementos valorizadores (piscina, garagem), patologias (trincas, infiltrações). Por exemplo, ao detectar “fissura” o sistema acrescenta tag relevante. O Roboflow destaca que YOLO é rápido e leve, “fazendo uma só passagem no *network* para classificar caixas delimitadoras” ⁶ – ideal para análises em tempo real. O resultado dessas detecções alimenta o questionário dinâmico (p.ex., se foi detectada piscina, perguntar estado da mesma).
- **Geração de Questionário Dinâmico:** o back-end coleta as características extraídas (endereços, fotos, respostas iniciais) e monta prompts para LLM. Utilizar **LangChain** em Python para facilitar essa cadeia: por exemplo, um *chain* que primeiro chama modelo para gerar perguntas específicas (“Detectamos armários planejados na cozinha; estão em bom estado?”), e se necessário o modelo LLM cria perguntas follow-up baseadas em respostas do usuário. ADAsci demonstra que uma aplicação de “question generator” pode ser construída eficientemente usando Python, Pydantic e LangChain ⁷. As perguntas são então apresentadas no front-end (múltipla escolha ou texto livre).
- **Metodologia de Avaliação:** ao final das respostas, determinar método de avaliação segundo NBR14653. Por padrão, usar **Comparativo Direto de Dados de Mercado** (mínimo ~6 comparáveis confiáveis). Caso haja poucos dados, recorrer a Método Evolutivo (terreno + custode reposição) ou Método da Renda (para imóveis locados). Esse motor de decisão analítico pode ser implementado em Python: basta verificar quantidade/qualidade de dados coletados e critérios normativos (ex.: “se >=6 comparáveis válidos, usar comparativo”etc.), registrando justificativa no laudo.
- **Tratamento Estatístico:** usar Python (pandas, NumPy, statsmodels ou scikit-learn). Por exemplo, ajustar regressão linear ou hedônica para calcular valor por m². Realizar homogeneização: ajustar área e padrão dos comparáveis para o imóvel avaliado. Cálculo do intervalo de confiança e grau de precisão também pode usar `scipy.stats`. Conforme PixelBrainy, scikit-learn é padrão para ML em Python ¹².
- **Geração do Laudo:** montar documento PDF/A-3 segundo NBR14653-1. Pode-se gerar HTML com Jinja2 (template) e converter via `wkhtmltopdf` ou usar ReportLab/LaTeX. O laudo final incluirá: identificação do solicitante, características do imóvel, análise de mercado, metodologia, memória de cálculo, valor, fotos legendadas, declarações. Bibliotecas como Python-PDFKit facilitam converter HTML estilizado em PDF. Para assinar digitalmente (ICP-Brasil), integrar ferramenta de assinatura (ex.: `certificate library` em Java/Python ou serviço externo) e incorporar carimbo temporal (blockchain ou timestamp).

4. Integrações e Infraestrutura

- **APIs Externas:** Google Maps ou OpenStreetMap (Leaflet) para mapas; Nominatim para geocoding; APIs de transações imobiliárias ou portais (via REST ou scraping). Qualquer API paga deve ser avaliada quanto a custo (ex.: limites do Google Maps).

- **CI/CD e DevOps:** usar Git, GitHub/GitLab. Pipeline de testes automatizados (unitários em Python/JS) e deployment contínuo via Docker Compose ou Kubernetes. O monitoramento pode usar ferramentas como Sentry ou Prometheus.
- **Armazenamento e Segurança:** criptografar dados sensíveis (ex.: CNPJ do avaliador). Seguir LGPD para dados de usuários. Manter backups regulares do banco.
- **Auditoria e Versionamento:** registrar todas as entradas/decisões do IA (logs de prompts e respostas, versão do modelo) para rastreabilidade. O laudo final e datasets brutos devem ser arquivados (por exemplo, 20 anos conforme recomendação de perícias judiciais), garantindo reproduzibilidade.

5. Fluxo de Desenvolvimento (Detalhado)

1. **Configuração Inicial:** criar repositório Git. Instalar Node.js e/ou Python3. Configurar virtualenv ou Dockerfile base. Utilizar ferramentas de apoio: o Gemini CLI (open-source) pode acelerar tarefas (ex.: criação de componentes, correção de código) diretamente no terminal ⁹. Da mesma forma, o Claude Code (requisição paga) pode mapear o *codebase* e automatizar edições complexas ¹⁰.
2. **Protótipo de Geocoding:** implementar serviço que recebe endereço e retorna lat/long, zona e infraestrutura do entorno. Exemplo: usar `geopy` em Python para chamar Nominatim, armazenando resultado em PostGIS. Verificar leis de zoneamento (via API ou dados públicos). Testar consultas espaciais básicas (Ex.: propriedades <1km usando `ST_DWithin`) para validar estrutura espacial ⁴.
3. **Ingestão de Fotos e Análise:** criar API `/analyze/images` que salva as fotos e executa detecção de objetos. Por exemplo, integrar o modelo YOLOv8 (Ultralytics) com PyTorch, passando cada imagem e recebendo labels (por ex. `{"piscina": true, "fissura": false, ...}`). Salvar essas *tags* no banco e enviar para o módulo de geração de perguntas.
4. **Questionário Inteligente:** construir prompt base (incluindo informações já coletadas) e usar LLM. Por exemplo, em Python:

```
prompt = f"Imóvel em {cidade}, detectamos piscina: {detectouPiscina}...  
Considerando NBR14653, quais perguntas múltipla escolha fazer para  
fundamentar a avaliação?"  
response = langchain.llm_chain.run(prompt)
```

Ajustar prompt conforme feedback. O resultado (lista de perguntas) é enviado ao front-end e apresentado ao usuário.

5. **Processamento das Respostas:** após o usuário responder o questionário, enviar os dados a `/evaluate/insights`. Nesse ponto, fazer filtragem e tratamento estatístico: calcular preço médio/m² dos comparáveis, aplicar ajuste linear (scikit-learn), e computar grau de fundamentação. Por exemplo, verificar se R^2 e desvio padrão atendem NBR (ex.: $GP \leq 10\%$). Em caso negativo, realimentar coleta de dados ou alertar usuário para fornecer mais informações.
6. **Geração do Laudo Preliminar:** usar template LaTeX ou HTML para criar rascunho do laudo. Incluir automaticamente: dados do solicitante, fotos (legendadas conforme tags detectadas), tabela de comparáveis (com fotos e características), memória de cálculo e conclusões. Todos os itens obrigatórios da NBR14653-1 devem ser preenchidos.
7. **Revisão Manual Assistida:** permitir que o perito logado revise cada seção: por ex., editar descrições, substituir imagens, ou ajustar valores. Cada alteração recalcula automaticamente estatísticas-chave (como GP). Ferramenta de *diff* pode mostrar mudanças.

8. **Finalização e Assinatura:** ao aprovar o laudo, converter para PDF/A-3 com XML embutido. Assinar digitalmente (ICP-Brasil) e adicionar timestamp público (blockchain privada ou registro oficial) para garantias de integridade. Enviar ao usuário link para download e QR-code de verificação.
9. **Documentação e Entrega:** gerar um pacote contendo PDF final, planilha Excel (se cabível) dos cálculos, base de dados anonimizados e certificado de conformidade. Isso provê transparência adicional.

Cada etapa acima exige testes unitários (ex.: validade das consultas espaciais, precisão do OCR/ML, respostas do LLM). A equipe de desenvolvimento deve documentar as APIs e o **prompt engineering** empregado. Em produção, monitorar métricas de precisão (como **Grau de Precisão** estatístico) e feedback de usuários para aprimoramento contínuo.

Fontes: As escolhas de tecnologia baseiam-se em tendências atuais: o uso de React/Node e Python para IA é comum ¹ ³; frameworks Python como Django/FastAPI facilitam prototipagem ²; a biblioteca Leaflet lidera para mapas; PostGIS reduz complexidade de consultas espaciais ⁴; LangChain e OpenAI são recomendados para IA de perguntas ⁷ ⁸; e ferramentas CLI IA (Gemini, Claude) aceleram a codificação ⁹ ¹⁰. Este guia técnico detalha todas as etapas de implementação de acordo com as normas NBR 14653, visando ágil desenvolvimento e robustez de fundamentação.

¹ ¹² A Guide to Real Estate AI Software Development in 2025

<https://www.pixelbrainy.com/blog/real-estate-ai-software-development-guide>

² Top Web Technologies For Web Development In 2025

<https://www.tekrevol.com/blogs/top-web-technologies-for-web-development/>

³ ⁸ Top 10 Tech Stacks for Software Development in 2025

<https://www.imaginarycloud.com/blog/tech-stack-software-development>

⁴ ⁵ The Hidden Superpowers of PostgreSQL: Geospatial Queries | Dennenboom Blog

<https://dennenboom.be/blog/the-hidden-superpowers-of-postgresql-geospatial-queries>

⁶ What is YOLO? The Ultimate Guide [2025]

<https://blog.roboflow.com/guide-to-yolo-models/>

⁷ Building GenAI-Based Dynamic Question Generator from Scratch

<https://adasci.org/building-genai-based-dynamic-question-generator-from-scratch/>

⁹ Google announces Gemini CLI: your open-source AI agent

<https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/>

¹⁰ Claude Code: Deep Coding at Terminal Velocity \ Anthropic

<https://www.anthropic.com/claude-code>

¹¹ Building LLM-Powered Real Estate Intelligent Agents: Technical Implementation and Business Value - DEV Community

<https://dev.to/jamesli/building-llm-powered-real-estate-intelligent-agents-technical-implementation-and-business-value-5dbj>